

Coordinate Transformation of CAD Drawings in Portugal

Geodetic foundations, DGT guidance and implementation with PT-TM06/ETRS89 (EPSG:3763)

Software: CAD EPSG Converter · Main application: `script.py` · Version: 1.0

Abstract

The transformation of an engineering drawing between coordinate reference systems requires more than changing an EPSG identifier or translating its origin. It combines the interpretation of the source reference system, inverse cartographic projection, a datum transformation where necessary, forward projection and the reconstruction of CAD geometry. This paper documents a Python application for DXF/DWG reprojection, with particular attention to mainland Portugal and the target system PT-TM06/ETRS89, identified by EPSG:3763. It explains the geodetic relationships between ETRS89, Datum 73, Datum Lisboa, ED50 and the reference systems of the Portuguese archipelagos. The mathematical discussion covers ellipsoidal and geocentric coordinates, Transverse Mercator projection, Helmert and Molodensky transformations, polynomial models, NTV2 displacement grids, interpolation, uncertainty and geometric approximation. The implementation uses `pyproj`/PROJ for coordinates, `ezdxf` for DXF processing and an external ODA File Converter for DWG exchange. Particular emphasis is placed on distinguishing numerical precision, datum-model accuracy and CAD representation fidelity. Reproducible operating procedures, Windows installation commands, standalone packaging instructions and a bibliography complete the document.

Keywords: coordinate reference system; EPSG:3763; PT-TM06; ETRS89; Datum 73; Datum Lisboa; DGT; NTV2; CAD; DXF; DWG; geodetic transformation.

1. Purpose and scientific scope

1.1 The engineering problem

A CAD coordinate is a number with an implied spatial meaning. The point (10000, 10000) may represent metres in a national projection, coordinates in a local construction grid, or an arbitrary position in a drawing. Its numerical form does not identify its datum, projection, axis order or units. A reliable transformation therefore begins with documented source metadata.

The application is intended for drawings whose coordinates already belong to one of its supported reference systems. Its principal workflow is **EPSG:27493, Datum 73 / Modified Portuguese Grid, to EPSG:3763, ETRS89 / Portugal TM06**. The user may select other supported source and target systems, including reverse transformations. Merely assigning the target EPSG code to an unchanged drawing is not equivalent to converting its coordinates.

In geodetic terminology, a *coordinate conversion* changes representation without changing datum, such as geographic to projected coordinates. A *coordinate transformation* changes datum through an empirically established relationship. A CAD reprojection can contain both operations. [IOGP, Guidance Note 7-2](#)

1.2 Implementation scope

Item	Behaviour
Main file and version	<code>script.py</code> , version 1.0
Default source CRS	EPSG:27493
Default target CRS	EPSG:3763
Native drawing processing	DXF with <code>ezdxf</code>
DWG processing	External ODA File Converter, with temporary DXF intermediates
Coordinate engine	<code>pyproj</code> , using PROJ and its CRS database
Interface	English Tkinter GUI and command-line interface
Output choices	DXF or DWG; one selected format for the complete batch
Main report	<code>report_3763.json</code> , or <code>report_<target EPSG>.json</code>
Source identification	Declared by the user; not inferred automatically
Format-only conversion	Identical source and target EPSG codes are rejected

Column key (left to right): Item; Application behaviour.

The mathematical treatment below explains both implemented operations and relevant background methods. The application does not expose every formula discussed as an independently selectable calculator. In particular, it does not implement a dedicated DGT polynomial calculator, a geoid conversion, a survey-network adjustment or a coordinate-epoch interface.

2. Reference systems in Portugal

2.1 DGT framework and territorial scope

The Direção-Geral do Território (DGT), Portugal's Directorate-General for Territory, treats mainland Portugal, the Azores and Madeira separately in its reference-system documentation. Horizontal/cartographic systems and vertical systems are also distinguished. The mainland vertical reference is **Cascais Helmert 1938**, established from the Cascais tide-gauge record from 1882 to the end of 1938. The archipelagos use island-specific vertical references. [DGT, Reference systems](#)

For an engineering drawing, this distinction has a direct consequence: selecting EPSG:3763 establishes a horizontal reference system, but does not establish that the drawing's Z coordinates are orthometric heights, ellipsoidal heights or elevations relative to a harbour datum. Those questions require separate source documentation.

2.2 PT-TM06/ETRS89: the main target

DGT states that PT-TM06/ETRS89 should replace the historical mainland systems. Its description relates ETRS89 to ITRS at epoch 1989.0 and to the stable Eurasian plate, and identifies the 1989, 1995 and 1997 international campaigns followed by GPS observation and adjustment of the mainland network. The PT-TM06 projection parameters apply from 2006. DGT also reports the EuroGeographics recommendation of Transverse Mercator for mapping at scales larger than 1:500,000. [DGT, PT-TM06/ETRS89](#)

The EPSG name is **ETRS89 / Portugal TM06**; **PT-TM06/ETRS89** is the familiar Portuguese system designation. The application uses **EPSG:3763** for this system.

Table 1. Projection and ellipsoid parameters for the principal target and source. Values are consistent with the DGT definitions and the EPSG records used by the application. [DGT, PT-TM06/ETRS89](#); [DGT, Datum 73](#)

Parameter	EPSG:3763	EPSG:27493
Datum/reference system	ETRS89	Datum 73
Ellipsoid	GRS80	International 1924 / Hayford
Semi-major axis, m	6,378,137	6,378,388
Inverse flattening	298.257222101	297
Projection family	Transverse Mercator	Gauss–Krüger / Transverse Mercator
Latitude of natural origin	39°40′05.73″ N	39°40′00″ N
Longitude of natural origin	8°07′59.19″ W	8°07′54.862″ W
Latitude of origin, decimal degrees	39.668258333333	39.666666666667
Longitude of origin, Greenwich degrees	−8.133108333333	−8.131906111111
Central-meridian scale factor	1.0	1.0
False easting, m	0.000	+180.598
False northing, m	0.000	−86.990
Application X/Y convention	Easting / northing	Easting / northing

Column key (left to right): Parameter; EPSG:3763, target; EPSG:27493, source.

Negative coordinates are valid in systems with an origin inside the territory. A negative easting or northing is not evidence of a conversion failure. Conversely, plausible-looking positive values do not prove that the correct source datum has been selected.

2.3 Datum 73 and the historical mainland networks

Datum 73 followed reobservation of the mainland geodetic network in the 1970s. DGT identifies Melriça TF4 as its origin and describes the central location and single-block first-order

adjustment. The system uses the Hayford ellipsoid and the false-origin values shown in Table 1. [DGT, Datum 73](#)

The origin offsets are part of its projection definition. Subtracting 180.598 m from X and adding 86.990 m to Y cannot, by itself, transform Datum 73 to ETRS89: the datum relationship and the target projection must also be applied.

Datum Lisboa has a different network history. DGT also describes the **Local Lisbon Triangulation**, established for 1:1,000 mapping, with a tangent-plane representation and coordinates of 12,000 m and 6,000 m assigned to Castelo de São Jorge. A drawing described only as “Lisbon coordinates” may therefore require more investigation than choosing a national EPSG code. [DGT, Datum Lisboa](#)

The Bessel-based historical system needs particular care. DGT notes that the historical Bessel–Bonne coordinates in use are not rigorously Bonne coordinates: they were derived by polynomial transformation from Hayford–Gauss Datum Lisboa. Its M coordinate is west-positive and P is south-positive. The application interface for EPSG:2963 is **X = P, southing; Y = M, westing**. Historical source provenance and independent control points remain essential. [DGT, Bessel Datum Lisboa](#)

ED50 is a separate datum using International 1924, with its origin associated with Potsdam. The mainland projected representation included in this application is EPSG:23029, UTM zone 29N. Its transformation requires the ED50 relationship; neither a Datum 73 grid nor a Datum Lisboa grid is interchangeable with it. [DGT, ED50](#)

2.4 Azores and Madeira

DGT identifies PTRAO8-UTM/ITRF93 for the autonomous regions, linked to ITRF93 through the TANGO 1994 campaign and subsequent observations of the island networks. It recommends the UTM zone appropriate to the island group and replacement of the historical systems. [DGT, PTRAO8-UTM/ITRF93](#)

Table 2. Principal island projections.

Region	EPSG	UTM zone	Meridian
Western Azores	5014	25N	33° W
Central and eastern Azores	5015	26N	27° W
Madeira	5016	28N	15° W

Column key (left to right): Region; Application EPSG; UTM zone; Central meridian.

These projections use GRS80, latitude of origin 0°, false easting 500,000 m, false northing 0 m and central-meridian scale factor 0.9996. The mainland default EPSG:3763 should therefore not be treated as a universal projection for every Portuguese island. The four mainland historical-datum grids distributed with this workflow are not island transformation grids.

2.5 Vertical references and geoid heights

For a compatible geoid model, ellipsoidal height h , orthometric height H and geoid undulation N_g are related by:

$$H = h - N_g.$$

DGT describes GeodPT08 for mainland Portugal, with 0.025° grid spacing, GRS80-referenced undulations and an estimated overall vertical accuracy of 4 cm relative to the national geodetic and levelling networks. This is a distinct dataset and operation from a horizontal NTV2 grid. [DGT](#), [Geoid model](#)

The application does not apply the height relation above. In ordinary horizontal conversions, it preserves CAD Z by default. Keeping a value such as 12.350 unchanged preserves that number's existing vertical meaning; it does not convert it to Cascais Helmert 1938, chart datum or ellipsoidal height.

3. EPSG systems available in the application

The following tables cover all **27 identifiers** in the source and target selectors. The names are taken from the CRS definitions used by pyproj; some GUI descriptions are shortened. These tables describe the application's coordinate interface, which can differ from the native axis order published in an EPSG geographic CRS definition.

3.1 Mainland and European systems

EPSG	Reference system	Role
3763	ETRS89 / Portugal	Projected metres; main target;
27493	TM06 Datum 73 / Modified	X=easting, Y=northing Projected metres; default source
	Portuguese Grid	
4258	ETRS89	Geographic 2D; longitude/latitude in degrees
4937	ETRS89	Geographic 3D; longitude/latitude plus ellipsoidal height
4936	ETRS89	Geocentric XYZ in metres
4274	Datum 73	Geographic 2D
4207	Lisbon	Geographic 2D, Greenwich longitude; Lisbon 1937 datum
5018	Lisbon / Portuguese Grid New	Projected metres; zero false easting/northing
20790	Lisbon (Lisbon) / Portuguese National Grid	Projected metres; Lisbon prime meridian; 200,000/300,000 m false offsets

EPSG	Reference system	Role
20791	Lisbon (Lisbon) /	Projected metres; Lisbon prime
4666	Portuguese Grid Lisbon 1890	meridian; zero false offsets Geographic 2D; Bessel-based datum
2963	Lisbon 1890 (Lisbon) /	Projected metres;
4230	Portugal Bonne ED50	X=southing, Y=westing in this application Geographic 2D
23029	ED50 / UTM zone 29N	Projected metres; historical mainland ED50 data
25829	ETRS89 / UTM zone 29N	Projected metres; distinct from PT-TM06

Column key (left to right): EPSG; Reference system; Representation and principal role.

3.2 Autonomous regions

EPSG	Reference system	Role
5011	PTRA08	Geocentric XYZ in metres
5012	PTRA08	Geographic 3D
5013	PTRA08	Geographic 2D
5014	PTRA08 / UTM zone 25N	Projected metres; western Azores
5015	PTRA08 / UTM zone 26N	Projected metres; central/eastern Azores
5016	PTRA08 / UTM zone 28N	Projected metres; Madeira
2188	Azores Occidental 1939 / UTM zone 25N	Historical western Azores datum
2189	Azores Central 1948 / UTM zone 26N	Historical central Azores datum
2190	Azores Oriental 1940 / UTM zone 26N	Historical eastern Azores datum
2942	Porto Santo / UTM zone 28N	Historical Porto Santo datum

Column key (left to right): EPSG; Reference system; Representation and principal role.

3.3 Global exchange and web display

EPSG	Reference system	Role
4326	WGS 84	Geographic 2D; application X=longitude, Y=latitude

EPSG	Reference system	Role
3857	WGS 84 / Pseudo-Mercator	Web-map projected coordinates in metres

Column key (left to right): EPSG; Reference system; Representation and principal role.

EPSG:3857 is offered for interoperability. Its metre coordinate unit does not make its map distances suitable substitutes for local ground survey distances. ETRS89 and WGS 84 also should not be assumed identical for high-accuracy, epoch-sensitive work merely because their coordinates can appear similar at ordinary display precision.

3.4 Axis order and prime meridians

The application requests `always_xy=True`: geographic input is CAD X=longitude and CAD Y=latitude, in decimal degrees. A point near western mainland Portugal consequently has a negative X longitude and a positive Y latitude. Native EPSG geographic definitions commonly list latitude first, so importing a latitude/longitude table directly into CAD X/Y would reverse the coordinates. [pyproj](#), [Transformer API](#)

The Lisbon-meridian definitions in EPSG:20790 and EPSG:20791 are interpreted through their CRS definitions. The geographic hub used by the local grids has Greenwich-referenced longitude. The application performs the meridian conversion; the operator must not apply an additional manual longitude offset to already projected CAD coordinates.

EPSG:2963 is an explicit exception to the ordinary east/north interface. Its south/west axes remain part of the definition. The code constructs a compatible Bonne operation while retaining that axis convention and the Lisbon prime meridian. This mathematical handling does not remove the historical realisation qualification in Section 2.3.

4. Mathematical foundations

4.1 Notation

Symbol	Meaning	Usual unit
λ, φ	Longitude and geodetic latitude	Radians in formulas; degrees at the GUI/CLI geographic interface
E, N	Projected easting and northing	m
X, Y, Z	Geocentric Cartesian coordinates when explicitly identified as such	m
a, b	Ellipsoid semi-major and semi-minor axes	m
f, e^2	Flattening and squared first eccentricity	Dimensionless
ν, ρ	Prime-vertical and meridional radii of curvature	m

Symbol	Meaning	Usual unit
$h, H,$	Ellipsoidal height, orthometric	m
N_g	height and geoid undulation	
$P_s,$ P_t	Source and target projection operators	Coordinate mappings
$T_{s \rightarrow t}$	Datum transformation	Coordinate mapping
F, J_F	Complete coordinate mapping and its Jacobian	Mapping; derivative matrix
ε_s	Source-space curve faceting tolerance	Source drawing units

Angles in trigonometric equations are radians unless stated otherwise. A CAD entity's ordinary X/Y/Z properties must not be confused with Earth-centred Cartesian XYZ merely because the letters are the same.

4.2 Composition of a projected-to-projected transformation

For a source position $\mathbf{x}_s$, the general horizontal operation can be written:

$$\mathbf{x}_t = P_t(T_{s \rightarrow t}(P_s^{-1}(\mathbf{x}_s))).$$

The inverse source projection recovers geographic coordinates on the source datum. The transformation then expresses the position in the target datum. The target projection produces the required planar coordinates. Unit conversions and prime-meridian changes are included where needed. If the datums agree, the middle operation may reduce to an identity or a representation adjustment. [IOGP, Guidance Note 7-2](#)

For the principal application workflow, the chain is:

EPSG:27493 $\rightarrow$ inverse source projection $\rightarrow$ Datum 73 geographic coordinates $\rightarrow$ NTV2 grid $\rightarrow$ ETRS89 geographic coordinates $\rightarrow$ PT-TM06 projection $\rightarrow$ EPSG:3763.

The intermediate geographic positions are essential to applying the grid correctly. The NTV2 correction is not a constant translation to add directly to projected CAD X/Y.

4.3 Ellipsoid geometry and geocentric coordinates

The geometric relationships begin with:

$$f = (a - b)/a, \quad e^2 = 2f - f^2.$$

$$\nu = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}}, \quad \rho = \frac{a(1 - e^2)}{(1 - e^2 \sin^2 \varphi)^{3/2}}.$$

The forward geographic-to-geocentric conversion is:

$$X = (\nu + h) \cos \varphi \cos \lambda.$$

$$Y = (\nu + h) \cos \varphi \sin \lambda.$$

$$Z = [\nu(1 - e^2) + h] \sin \varphi.$$

The origin is at the Earth's centre; the X axis intersects the equator at Greenwich, Y intersects it at 90° E, and Z follows the north polar axis. Here h is ellipsoidal height. EPSG:4936 and EPSG:5011 require genuine three-dimensional Cartesian coordinates, not a conventional plan drawing with an arbitrary Z elevation. [PROJ, Geodetic to Cartesian conversion](#)

The inverse operation recovers latitude, longitude and ellipsoidal height using the selected ellipsoid. PROJ supplies these conversions; the application does not implement an independent geocentric solver.

4.4 Transverse Mercator and PT-TM06

Transverse Mercator is conformal: it preserves infinitesimal angles while scale varies spatially. It does not preserve every finite distance or area. The central scale, ellipsoid, natural origin and false coordinates jointly specify a particular projection. PT-TM06 and UTM belong to this family but use different parameters. PROJ documents a high-order Krüger formulation for its precise Transverse Mercator implementation. The application delegates the projection calculation to PROJ. [PROJ, Transverse Mercator](#)

For explanatory purposes, define $A = (\lambda - \lambda_0) \cos \varphi$, $T = \tan^2 \varphi$, $C = e'^2 \cos^2 \varphi$ and $e'^2 = e^2/(1 - e^2)$. The familiar leading terms illustrate the nonlinearity:

$$E \approx E_0 + k_0 \nu \left[A + \frac{(1 - T + C)A^3}{6} \right].$$

$$N \approx N_0 + k_0 \left[M(\varphi) - M(\varphi_0) + \nu \tan \varphi \left(\frac{A^2}{2} + \frac{(5 - T + 9C + 4C^2)A^4}{24} \right) \right].$$

The meridional distance is $M(\varphi) = \int_0^\varphi \rho(\theta) d\theta$. The two Transverse Mercator series above are deliberately truncated illustrations, not the numerical implementation or a substitute for a complete projection algorithm. Their powers of longitude difference explain why a drawing-wide translation cannot reproduce a general projection change. Classical cartographic development is available in Snyder's working manual. [Snyder, Map Projections](#)

For a short measured line, a local grid scale factor k relates ellipsoidal and projected distance approximately by $d_{\text{grid}} \approx k d_{\text{ellipsoid}}$. Ground-to-grid reduction also depends on height. Consequently, a transformed CAD distance is not automatically a ground distance, and a dimension text value should not be treated as a new survey measurement without checking its meaning.

4.5 Three- and seven-parameter datum relationships

A three-parameter model accounts for a translation between geocentric origins. A seven-parameter Helmert model adds three small rotations and one scale parameter. In position-vector convention, its component form is:

$$X_t = T_X + (1 + \mu)(X_s - r_Z Y_s + r_Y Z_s).$$

$$Y_t = T_Y + (1 + \mu)(r_Z X_s + Y_s - r_X Z_s).$$

$$Z_t = T_Z + (1 + \mu)(-r_Y X_s + r_X Y_s + Z_s).$$

If the published scale is s parts per million, then $\mu = s \cdot 10^{-6}$. Rotation values stated in arcseconds must be converted to radians before insertion into these equations. Coordinate-frame convention reverses the rotation signs. The convention and transformation direction therefore belong to the parameter specification. [DGT, Bursa–Wolf formula sheet](#); [PROJ, Helmert transform](#)

A national best-fit parameter set describes broad differences between reference frames. It cannot reproduce arbitrary local deformation within a historical triangulation network. A displacement grid provides spatially varying corrections to address that limitation.

A time-dependent transformation can additionally use $p(t) = p(t_0) + \dot{p}(t - t_0)$ for its parameters. The application has no coordinate-epoch input. It should not be described as an epoch-aware transformation service for high-precision dynamic reference frames. [PROJ, Helmert transform](#)

4.6 Molodensky transformation

The Molodensky formulation expresses approximate changes directly in geographic coordinates, using geocentric translations and differences between ellipsoids. With $\Delta a = a_t - a_s$ and $\Delta f = f_t - f_s$, define:

$$Q = -\Delta X \sin \varphi \cos \lambda - \Delta Y \sin \varphi \sin \lambda + \Delta Z \cos \varphi.$$

$$B = \Delta a \frac{e^2 \nu}{a} \sin \varphi \cos \varphi + \Delta f \left(\frac{a}{b} \rho + \frac{b}{a} \nu \right) \sin \varphi \cos \varphi.$$

$$\Delta \varphi = (Q + B) / (\rho + h).$$

$$\Delta \lambda = \frac{-\Delta X \sin \lambda + \Delta Y \cos \lambda}{(\nu + h) \cos \varphi}.$$

$$\Delta h = \Delta X \cos \varphi \cos \lambda + \Delta Y \cos \varphi \sin \lambda + \Delta Z \sin \varphi - \Delta a \frac{a}{\nu} + \Delta f \frac{b}{a} \nu \sin^2 \varphi.$$

Target coordinates are obtained by adding these changes to the source latitude, longitude and ellipsoidal height. These equations reproduce the mathematical structure of DGT's formula

sheet using the notation defined above. They require the source ellipsoid and consistent units. They are background theory here, rather than a separately exposed application mode. [DGT, Molodensky formula sheet](#)

4.7 Second-degree polynomial models

A planar polynomial can absorb smooth systematic differences through normalized source coordinates. To avoid confusing its normalization constants with ellipsoidal height or map scale, denote those constants by L_E and L_N :

$$u = (E - E_r)/L_E, \quad v = (N - N_r)/L_N.$$

$$E_t = a_0 + a_1u + a_2v + a_3u^2 + a_4uv + a_5v^2.$$

$$N_t = b_0 + b_1u + b_2v + b_3u^2 + b_4uv + b_5v^2.$$

DGT publishes this form together with fitted coefficient sets. The normalizing origin and scales are indispensable: coefficients cannot be transferred to unnormalized coordinates or to a different grid origin without altering the model. A polynomial is empirical and should not be extrapolated outside its intended system and region without validation. The present application uses its configured coordinate-operation chain rather than directly evaluating DGT's published polynomial tables. [DGT, Polynomial formula sheet](#)

5. Portuguese datum transformations and NTv2 grids

5.1 DGT grid construction and published validation

DGT distributes NTv2 grids for Datum 73 and Datum Lisboa to ETRS89. Its description specifies 1,129 GPS-observed national geodetic vertices, kriging to construct the grid, 72'' spacing, and validation against 130 additional vertices excluded from construction. The published results are shown below. DGT identifies this grid work as CC BY 4.0. [DGT, Mainland coordinate transformations](#)

Table 3. DGT's reported grid validation statistics.

Source datum	RMSE, m	Maximum error, m
Datum 73	0.06	0.16
Datum Lisboa	0.09	0.30

Column key (left to right): Source datum; Published RMSE, m; Published maximum absolute error, m.

These are dataset-validation statistics, not a guaranteed error for every input vertex. DGT also supplies polynomial, Molodensky and Bursa–Wolf alternatives; their parameters and reported residuals are available in its transformation parameter sheet. [DGT, Mainland coordinate transformations; IGP/DGT, Transformation parameters](#)

For reproducible mainland work, this README recommends using the DGT grids for these two datum families and recording the exact files. This is an implementation recommendation based on the official source and traceable validation; it is not a claim that DGT has certified this CAD application.

5.2 Published parameter methods in context

Table 4. Horizontal residual RMSE for DGT’s fitted parameter methods, in metres. Values are component statistics from the published parameter sheet; they are not directly interchangeable with a single combined horizontal error statistic.

Method	73 E	73 N	Lisboa E	Lisboa N
Degree-2 polynomial	0.106	0.096	0.793	0.852
Bursa–Wolf	0.381	0.359	1.404	1.493
Molodensky	0.844	0.563	1.694	1.600

Column key (left to right): Method; Datum 73 E; Datum 73 N; Datum Lisboa E; Datum Lisboa N.

The polynomial adjustment/check sets differ between datums; the parameter methods and NTv2 grids also have different validation datasets. Table 4 illustrates why local network deformation matters, rather than providing a universal ranking at every site. [IGP/DGT](#), [Transformation parameters](#)

Gonçalves’ CNCG 2009 paper independently discusses the grid approach, including 0.1° spacing and an independent sample of 147 geodetic points. Its reported longitude/latitude component standard deviations were 0.052/0.047 m for Datum 73 and 0.073/0.087 m for Datum Lisboa. The accompanying FCUP resource describes both its four-datum grid collection and the DGT grids. [Gonçalves, 2009](#); [Gonçalves, Coordinate transformations in Portugal](#)

The DGT spacing is finer, but comparing 0.1° and 0.02° alone does not establish which model is more accurate at a particular location. The control points, source observations, interpolation strategy and validation procedure also matter. A comparison of two models should use a common independent control dataset.

5.3 Grid construction and grid application are different operations

Kriging estimates a correction surface from irregularly distributed geodetic observations during grid preparation. The resulting NTv2 file stores values at regular nodes. During an ordinary conversion, PROJ reads those existing values and interpolates locally; it does not repeat the original network adjustment or kriging process.

For a point within a cell, let u and v be normalized cell coordinates between zero and one. Either correction component follows the bilinear rule:

$$\delta(u, v) = (1 - u)(1 - v)\delta_{00} + u(1 - v)\delta_{10} + (1 - u)v\delta_{01} + uv\delta_{11}.$$

The weights sum to one. At a corner the interpolated correction equals the stored corner value. Along an edge, the equation reduces to linear interpolation between the adjoining nodes. The

surface is piecewise bilinear, so denser storage samples the correction field more closely without creating new independent observations. [Gonçalves, 2009](#)

If corrections have already been expressed in an east-positive longitude convention:

$$\lambda_t = \lambda_s + \delta\lambda(\lambda_s, \varphi_s), \quad \varphi_t = \varphi_s + \delta\varphi(\lambda_s, \varphi_s).$$

NTv2 longitude offsets conventionally use west-positive values. The binary-format convention and the geographic-coordinate convention must therefore be interpreted together. PROJ handles the format-specific signs; users should not negate grid contents or manually reverse longitude signs. [PROJ, Geodetic TIFF grid conventions](#)

5.4 Inverse transformation and coverage

For $q_t = q_s + \vec{\delta}(q_s)$, an illustrative inverse iteration is:

$$q_s^{(j+1)} = q_t - \vec{\delta}(q_s^{(j)}).$$

The correction is defined as a function of source position. Subtracting one correction evaluated at the target position is generally not the rigorous inverse. The application requests PROJ's inverse grid operation when converting from ETRS89 to a historical datum. [IOGP, Guidance Note 7-2](#); [PROJ, Horizontal grid shift](#)

The application uses mandatory grid stages. Missing files, incompatible datum headers, loading failures and positions outside valid coverage stop the drawing conversion. Enabling ballpark transformations or allowing unresolved CAD objects does not bypass a required local grid.

5.5 Grid filenames required by script.py

The program discovers four specific filenames. The two DGT downloads must be supplied under the corresponding application names; their original download names are not detected automatically.

Table 5. Datum families, filenames and compatible DGT copies.

Datum	EPSG codes	Grid filename	DGT download
Datum 73	27493, 4274	pt73_e89.gsb	Copy D73_ETRS89_geo.gsb under this name
Lisbon 1937 / Hayford	4207, 5018, 20790, 20791	ptLX_e89.gsb	Copy DLX_ETRS89_geo.gsb under this name
Lisbon 1890 / Bessel	4666, 2963	ptLB_e89.gsb	Retain the separate Bessel grid
ED50	4230, 23029	ptED_e89.gsb	Retain the separate ED50 grid

Column key (left to right): Datum family; Relevant source/target EPSG codes; Filename read by application; Compatible DGT download.

Renaming a copy does not change its binary grid contents. The two DGT files supplied with this project pass the script's checks because their source identifiers are **DATUM73** and **DATUMLX**, their target is **ETRS89**, and their angular unit is **SECONDS**. Keep source downloads and attribution with the project record. Replacing these external files under the expected names does not require recompiling **script.exe**.

Table 6. Inspection of the supplied binary files. These values were read from the actual attachments, rather than inferred from filename size.

Grid set	Version	Spacing	Nodes/file	Bytes/file
DGT Datum 73 and Datum Lisboa files	IGP2011	72'' = 0.02°	59,010	944,528
Supplied pt73_e89.gsb and ptLX_e89.gsb files	JAG08_01	360'' = 0.1°	2,501	40,368

Column key (left to right): Grid set; Internal version; Angular spacing; Node count per file; Bytes per file.

The inspected DGT grids have 281 latitude rows and 210 longitude columns. Their rectangular node domain is approximately 36.7638889° to 42.3638889° N and −9.9305556° to −5.7505556° longitude. A rectangular file extent is not proof that offshore or border extrapolations have the quality of the control network.

5.6 Search paths, priorities and operation selection

Configuration	Search behaviour
NTv2 grid folder or --grid-dir specified	Search that directory only
Running script.exe without an explicit folder	Search the executable directory before the packaged module/data directories
Running script.py without an explicit folder	Search beside the script
Expected grid appears in more than one default location	First matching file wins
Application started from a different working directory	Working directory does not become an implicit grid location

An explicit grid folder replaces default search locations; missing files are not silently taken from another folder. Avoid commas, double quotation marks and line breaks in a grid directory path. Spaces are supported.

For different historical datum families, the chain passes through ETRS89 and uses the appropriate forward source grid and inverse target grid. Within one historical family, only the necessary projection and meridian operations are applied. For other supported CRS combinations, **TransformerGroup** supplies available PROJ operations. The script records the selected operation and warns if a better operation is unavailable. Ballpark operations are disabled by default. [pyproj](#), [Transformer API](#)

The local NTV2 stage converts geographic degrees to radians, applies **hgridshift**, and converts back to degrees. The geographic hub is ETRS89, with a three-dimensional representation used when a geocentric CRS participates. These are coordinate-operation stages; they do not alter CAD layer names or determine the original survey datum.

6. Reprojection of CAD geometry

6.1 A coordinate mapping is not a general CAD object mapping

Let F be the complete source-to-target coordinate mapping. Transforming the centre of a circle does not fully transform the circle, because scale and orientation can vary across its extent. Similarly, one shared block definition cannot generally represent the exact nonlinear image of every insertion at different positions.

The application consequently uses several geometric strategies. Point-defined entities are handled through their defining locations; curves are faceted; supported composite objects are exploded; and some entities use a local affine approximation. “Transformed coordinates” therefore does not mean that every original object type, constraint, width, payload or editing relationship is preserved exactly.

6.2 World coordinates and object coordinates

CAD entities can express geometry in a world coordinate system, an object coordinate system, or a block-local frame. Extrusion vectors, insertion matrices, rotations and scales affect the displayed positions. A transformation must act on the intended world positions rather than on local coordinates that happen to have similar numeric values.

The implementation uses **ezdxf** entity/path facilities to obtain world geometry where supported. SOLID and TRACE vertices are explicitly taken in world coordinates. Block and dimension handling inspects virtual geometry before explosion. These choices reduce coordinate-frame mistakes but do not turn an arbitrary proprietary object into fully accessible geometry.

6.3 Curves and source-space chord tolerance

For a circular arc of radius R and angular interval $\Delta\theta$, the maximum separation between arc and chord is:

$$e_c = R [1 - \cos(\Delta\theta/2)].$$

A chord-error requirement $e_c \leq \varepsilon_s$ gives, for $0 < \varepsilon_s < R$:

$$\Delta\theta \leq 2 \arccos(1 - \varepsilon_s/R).$$

For small angular intervals, $e_c \approx R\Delta\theta^2/8$. These elementary relationships explain why tighter faceting tolerances increase the vertex count. The application uses `ezdxf` path flattening rather than a circle-only formula, so the same user parameter can be applied to different supported curve types.

The default tolerance is **0.01 source drawing units** for a projected source. If those units are metres, it represents a centimetre-scale source-curve faceting criterion. The default for a geographic source is **0.000001 degrees**. A degree tolerance does not correspond to a uniform metre tolerance: its east-west distance varies with latitude.

After faceting, the vertices are transformed and connected with straight segments. A source-space tolerance is not a certified target-space error bound. In a small neighbourhood, its effect is related to the local derivative of F , while projection/grid curvature introduces an additional contribution. The software does not compute a global bound for that combined geometric error.

Straight LINE entities and straight portions between polyline vertices are **not adaptively densified for transformation curvature**. Their transformed endpoints are connected by straight segments. For very long segments, check intermediate positions independently or prepare suitably segmented source geometry before conversion.

6.4 Local affine approximation

For an anchor x_0 and a nearby displacement Δx :

$$F(x_0 + \Delta x) \approx F(x_0) + J_F(x_0)\Delta x.$$

This first-order expansion captures local translation, rotation, scale and shear. The neglected term grows with both object extent and spatial variation of the derivative. A single affine matrix therefore cannot guarantee an exact transformation of a large raster frame, text object, proprietary solid or other extended entity under a nonlinear mapping.

The script estimates the Jacobian with finite differences. The derivative step is 0.00001 degrees for geographic sources and `max(0.001, curve_tolerance)` in source units otherwise. This step is a numerical derivative parameter, not a grid accuracy statement. The report lists objects that received local affine treatment.

6.5 Entity treatment

Table 7. CAD processing strategies and their consequences.

Entity group	Processing	Interpretation
LINE, POINT, 3DFACE	Transform defining locations	Straight edges remain straight between transformed vertices
SOLID, TRACE	Transform world-coordinate vertices	Entity representation is normalized to the global extrusion direction
ARC, CIRCLE, ELLIPSE, SPLINE,	Facet source path, transform vertices,	Original analytical curve type is replaced

Entity group	Processing	Interpretation
HELIX LWPOLYLINE and ordinary 2D POLYLINE	create polyline Flatten path, including bulges	Bulges become straight polyline segments; widths/thickness are not guaranteed
3D POLYLINE, polygon mesh, polyface mesh	Transform stored vertex locations	Existing topology/vertex relationships are retained where supported
MESH	Transform vertices	Mesh structure is retained
LEADER	Transform vertices	Leader geometry moves; application-specific annotation semantics may differ
HATCH, MPOLYGON	Transform faceted boundary loops	Pattern definition stays local; invalid/nonplanar results stop publication
INSERT	Preflight, explode, recursively transform displayed geometry	Shared block and parametric editing semantics are not retained for exploded references
DIMENSION	Preflight rendered geometry, explode, transform components	Dimension text is not recalculated as a new engineering measurement
TEXT, MTEXT and other transformable entities	Local Jacobian/affine treatment	Local approximation is recorded
VIEWPORT, if paper-space conversion is enabled	Local affine treatment	Viewport/display relationships require inspection
Raster images, underlays and OLE content	Only accessible geometry/placement can be handled	Pixel data and embedded payloads are not warped
External reference files	No independent recursive file transformation	Referenced drawings require their own managed conversion
GEODATA/XRECORD and other document	Retained where supported	Embedded coordinates are not

Entity group	Processing	Interpretation
metadata		universally reprojected

Column key (left to right): Entity group; Processing strategy; Consequence requiring interpretation.

Constant-elevation curve replacements generally become LWPOLYLINE entities. Varying-elevation or geocentric replacements use 3D POLYLINE entities. General graphic attributes, such as layer, colour and lineweight, are copied to replacements where supported; XDATA is copied where possible and source-entity provenance is added. This is not a promise of byte-for-byte preservation of all CAD data.

6.6 Composite preflight and strict mode

Before an INSERT or DIMENSION is destructively exploded, the application inspects its available geometry. Missing block definitions, recursive references, content that cannot be copied completely, inaccessible dimension geometry and XCLIP boundaries are treated as unresolved conditions. This avoids silently accepting a drawing from which unsupported block contents have disappeared.

The default **Stop if a geometric object cannot be transformed** option prevents publication of a drawing containing unresolved geometry. It still permits the documented curve faceting and local affine approximations. Strict mode is therefore a completeness gate for identified failures, not a proof of exact geodetic or semantic preservation.

With strict mode disabled, a composite rejected during preflight may remain in its original, untransformed coordinates. Such an output can contain mixed reference systems. A handler failure that might have partly modified an object always stops publication of that drawing, even in non-strict mode.

Paper-space geometry is excluded by default. Retained block definitions, layout objects and coordinate-bearing metadata should not be described as universally reprojected simply because the model-space geometry was processed.

6.7 Z-coordinate behaviour

Coordinates	Behaviour
Purely horizontal 2D source/target; Keep Z enabled	Preserve the source Z value
Purely horizontal 2D source/target; Keep Z disabled	Set transformed Z to zero
Either selected CRS has three or more axes	Pass XYZ through the coordinate operation; the Keep Z checkbox does not override it
Horizontal NTV2 correction	Supplies no geoid or vertical-datum

Coordinates	Behaviour
	correction
Geocentric source or target	Requires meaningful three-dimensional coordinates

Column key (left to right): Coordinate situation; Actual behaviour.

An NTV2 horizontal grid also does not determine the change in ellipsoidal height between a historical datum and ETRS89. Passing XYZ through the coordinate chain cannot supply that missing relationship. Geocentric work requires heights consistent with the applicable ellipsoid and datum; unchanged historical ellipsoidal height is not automatically a rigorous ETRS89 height.

After conversion, a projected target is marked with CAD \$INSUNITS=6, meaning metres. Geographic and geocentric targets receive \$INSUNITS=0, with CRS provenance in custom header variables. Geocentric XYZ values are still numerically in metres; the unitless header is an application convention rather than a change of geocentric units.

7. Accuracy, uncertainty and verification

7.1 Separate sources of error

Table 8. Quantities that should not be confused.

Quantity	Meaning	Limitation
Floating-point precision	Numerical representation and arithmetic	Accuracy of the original survey
Coordinate-operation numerical error	Numerical evaluation of a specified mapping	Correct choice of source datum
Grid spacing	Sampling interval of a stored displacement field	Independent observation density or a universal error limit
Grid validation statistics	Agreement at a stated test dataset	Guaranteed accuracy at every drawing location
Curve tolerance	Source-geometry faceting	Datum-transformation uncertainty
Local affine approximation	First-order transformation of an extended object	Exact nonlinear distortion throughout that object
DXF/DWG audit and entity counts	Readability and structural checks	Geodetic accuracy or preservation of all object semantics
Round-trip closure	Internal forward/inverse consistency	Independent agreement with surveyed control

Column key (left to right): Quantity; What it describes; What it does not establish.

For independent uncertainty contributions, a first-order covariance model is:

$$C_t \approx J_F C_s J_F^T + C_{\text{op}}.$$

Here C_s represents uncertainty of the source coordinates and C_{op} represents the operation model. Correlations require additional terms and cannot simply be discarded. This equation is an interpretation framework; the application does not estimate these covariance matrices. [JCGM, Guide to the Expression of Uncertainty in Measurement](#)

For a simplified one-dimensional independent-error illustration, source and transformation standard uncertainties (one-standard-deviation equivalents) of 0.03 m and 0.06 m give $\sqrt{0.03^2 + 0.06^2} \approx 0.067$ m. This illustrative calculation does not turn DGT's published RMSE into a site-specific uncertainty estimate or include CAD approximation error.

7.2 Independent control points

Select control points with a documented source position and an independently established target position. Points distributed around the drawing extent are more informative than several points concentrated in one corner. Check the datum, vertical interpretation, coordinate epoch where relevant, and the independence of the reference values before comparing results.

For point i , define residuals $v_{E,i} = E_{i,\text{calc}} - E_{i,\text{ref}}$ and $v_{N,i} = N_{i,\text{calc}} - N_{i,\text{ref}}$. Useful summaries include:

$$\begin{aligned} \text{RMSE}_E &= \sqrt{\frac{1}{n} \sum_{i=1}^n v_{E,i}^2}, & \text{RMSE}_N &= \sqrt{\frac{1}{n} \sum_{i=1}^n v_{N,i}^2}. \\ \text{RMSE}_{2D} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (v_{E,i}^2 + v_{N,i}^2)}. \end{aligned}$$

Also examine mean residuals, the largest horizontal residual, the spatial pattern and any suspect control marks. A constant offset may indicate an origin issue; a rotated or position-dependent pattern can indicate axis, projection, datum or source-network problems. Residual interpretation requires the actual project context.

The software does not collect control-point pairs or automatically calculate these project validation statistics. Its JSON report documents the operation and CAD processing, while survey validation is a separate task.

7.3 Reproducible numerical example

Table 9. Illustrative coordinate calculations, EPSG:27493 to EPSG:3763. These are synthetic test coordinates, not surveyed accuracy-control points. Values were calculated using `pyproj` 3.7.2, `PROJ` 9.5.1 and the supplied DGT Datum 73 file under its compatible application alias. All coordinates are metres; horizontal conversion preserves Z.

Point	Axis	Source	Target
1	E	0.000000	0.343349
1	N	0.000000	−0.017373
1	Z	10.000	10.000
2	E	10000.000000	10000.017434
2	N	10000.000000	10000.039341
2	Z	25.000	25.000
3	E	−50000.000000	−50001.297927
3	N	100000.000000	99997.629952
3	Z	40.000	40.000

Column key: each numbered point retains its source and target E, N and Z coordinates from the same original coordinate tuple.

The SHA-256 of the Datum 73 file used for this example is:

54256060b00910d614fcf7d73c1c2514c90e6b389c750b6bfde46f7220358708

Unrounded coordinates returned to the source system with maximum component discrepancies below 2×10^{-9} m in this small calculation. This demonstrates numerical forward/inverse closure only. It is not evidence of nanometre survey accuracy, and applying the inverse to the rounded table values introduces additional rounding error.

7.4 File verification before publication

The application writes a temporary DXF, reopens it, audits it and compares its model-space entity count with the expected result. For DWG output, ODA converts that DXF to DWG, then converts the DWG back to an isolated DXF for another audit and count comparison. Publication follows these checks.

The output entity count need not equal the original count: exploding blocks and replacing curves changes the entity structure. The comparison concerns the expected processed document and its saved representation. Count equality alone cannot detect every coordinate error, changed text meaning, altered hatch appearance or unsupported application payload.

8. Application workflow and graphical interface

8.1 Prepare the drawing and reference information

Identify the source EPSG code from survey records, project specifications or reliable control points. Confirm that the coordinates are in the units expected by that CRS. A CAD drawing expressed in millimetres must not be passed as metre-based national coordinates merely because its CAD unit setting can be changed. Changing \$INSUNITS is not a rescaling operation.

Place the required grid files beside the script/executable, or select a complete grid folder in Advanced settings. For the normal Datum 73 to PT-TM06 conversion, the required local grid is `pt73_e89.gsb`. Keeping all four correctly identified files available supports the other historical families without repeated folder changes.

Install ODA File Converter when either the input or selected output is DWG. A DXF-to-DXF job does not require ODA.

8.2 Main controls

Table 10. Main GUI controls.

Control	Use
Open DXF / DWG...	Add one or several drawings
File / Type / Size list	Review inputs and select the drawing shown in the preview
Remove	Remove selected drawings from the batch
Clear	Remove all inputs
Source EPSG	Declare the actual coordinate reference system of the inputs
Target EPSG	Select the required output reference system
Validate transformation	Build/inspect the coordinate operation and required grids
Folder / Browse...	Select the output directory
Format	Select DXF or DWG for the whole batch
Suffix	Set the addition to each input stem, normally <code>_EPSG3763</code>
Convert drawings	Start the batch
Cancel	Request cooperative cancellation
Advanced...	Open numerical, data-handling and dependency settings
Preview / Log / Method	Inspect the input drawing, processing messages
/ help	and method notes

The interface reserves a left input/settings panel and a right preview/output area. Controls can scroll when the available work area is small, including a 1366×768 desktop. Text sizing and layout respond to the available screen area rather than assuming one fixed large monitor.

8.3 Output format selection

Opening or selecting a DWG chooses **DWG (.dwg)**. Opening or selecting a DXF chooses **DXF (.dxf)**. The user can then change the selection manually. The selected format applies to **every drawing in the current batch**, including a batch with mixed input formats.

Every drawing in a batch must share the selected source EPSG and coordinate-unit convention. Mixed DWG/DXF formats are supported, but drawings in different source reference systems require separate batches. Source, target and numerical settings are common to the batch.

Selecting another input can change the selected output format again. Reloading the preview does not undo a manual format choice. Before starting a mixed batch, check the Format field after making the final input selection. There is no “keep each input format” option.

Changing the target EPSG resets the suggested suffix to `_EPSG<target>`. If a custom suffix is needed, set it after the final target selection. The application rejects planned output names that collide with an input or with another planned output.

8.4 Preview interpretation

The preview displays the **original selected input**, in model-space top view, on a black background. It is not an automatic transformed-output view. Use the mouse wheel to zoom, drag with the left mouse button to pan, and **Fit** or a double-click to show the extents. **Reload** reads the selected input again.

The renderer respects supported layer visibility, colours, blocks, text, curves and hatch boundaries. Raster images are represented by frames. For responsiveness, display processing is bounded to 35,000 primitives and 300,000 vertices and uses its own display approximation. Notices identify preview limits or unsupported display content. These display limits do not reduce the conversion's processing scope.

The preview requires Pillow (PIL) as well as the ezdxf drawing frontend and Tkinter. Section 10 provides the complete installation and import checks. DWG preview also requires ODA to obtain a temporary DXF representation. The packaged build must collect these dependencies before it is distributed.

8.5 Advanced settings

Table 11. Advanced settings and defaults.

Setting	Default	Interpretation
Maximum curve chord error	0.01 projected source units; 0.000001° for the standard geographic default	Source-space faceting criterion
Keep Z elevations in horizontal conversions	On	Preserve Z in ordinary 2D horizontal operations
Transform paper-space geometry	Off	Include layout geometry only when explicitly intended
Stop if a geometric object cannot be transformed	On	Block drawings with unresolved geometry
Allow ballpark datum transformations	Off	Permit available approximate PROJ operations outside mandatory local-grid stages
Audit and recover damaged DXF	On	Attempt supported recovery and audit
Overwrite existing output files	Off	Require explicit choice before replacing CAD outputs
ODA File Converter executable	Automatic detection	Optional explicit executable path
NTv2 grid folder	Default search	An explicit directory is

Setting	Default	Interpretation
	locations	exclusive
Requirements	Information button	Show installation guidance

Validate transformation confirms that an operation can be constructed and its required grids loaded. It does not scan every drawing coordinate, establish full grid coverage or check survey-control accuracy. Coverage is checked when coordinates are actually transformed.

8.6 Typical GUI procedure

1. Start `script.py` or `script.exe`.
2. Open the drawing and inspect the source preview.
3. Confirm **Source EPSG:27493** and **Target EPSG:3763**, if those are the documented systems.
4. Choose the output folder and inspect the automatically selected output format.
5. Confirm the grid selection with **Validate transformation**.
6. Review Advanced settings, particularly the curve tolerance, Z policy and strict mode.
7. Select **Convert drawings** and follow the Log tab.
8. Open the completed drawing in the intended CAD application and review `report_3763.json`.
9. Compare relevant control points, dimensions, layers, hatches and important complex objects.

9. Command-line operation

Run without input paths to open the GUI. When input paths are supplied, `--output-dir` is required. The examples below use **Windows Command Prompt** and the virtual environment created and activated in Section 10. Run `call .venv\Scripts\activate.bat` in the project folder before using these examples.

9.1 Basic examples

Default Datum 73 to PT-TM06 conversion:

```
python script.py "survey.dxf" --output-dir "output"
```

DWG input with DXF output:

```
python script.py "survey.dwg" --output-dir "output" --format dxf
```

DXF input with DWG output and an explicit grid folder:

```
python script.py "survey.dxf" --output-dir "output" ^
--format dwg --grid-dir "grids"
```

A batch with an explicit common output format:

```
python script.py "site_a.dwg" "site_b.dxf" ^
--output-dir "output" --format dxf
```

Reverse conversion:

```
python script.py "survey_EPSG3763.dxf" ^
  --output-dir "reverse" ^
  --source-epsg 3763 --target-epsg 27493
```

If `--format` is omitted, the first input's extension chooses the output format for the entire batch. Quote paths that contain spaces. A relative `--grid-dir` is resolved from the command's working directory; this is an explicitly supplied path, distinct from the default grid search.

9.2 Complete option reference

Argument or option	Default / effect
inputs	One or more DXF/DWG paths; no inputs opens the GUI
<code>--output-dir</code>	Required for command-line conversion
<code>--source-epsg</code>	27493
<code>--target-epsg</code>	3763
<code>--format {dxf,dwg}</code>	First input's format if omitted
<code>--suffix</code>	_EPSG<target>
<code>--curve-tolerance</code>	0.000001 for geographic sources; 0.01 otherwise
<code>--transform-paper-space</code>	Include paper-space geometry
<code>--allow-ballpark</code>	Permit available ballpark operations; does not bypass mandatory local grids
<code>--allow-unresolved</code>	Permit untouched preflight-rejected composites; see Section 6.6
<code>--overwrite</code>	Permit replacing existing CAD outputs
<code>--oda</code>	Explicit ODA executable path
<code>--grid-dir</code>	Exclusive NTv2 directory
<code>--version</code>	Display version 1.0
<code>-h, --help</code>	Display command-line help

Keep-Z and DXF audit/recovery are enabled for normal CLI jobs; they have no CLI toggles. The internal ODA timeout is 900 seconds per converter invocation and is not exposed as a GUI/CLI setting. The Python `ConversionJob` interface contains these additional fields.

Ordinary completion returns exit code 0; handled conversion failure returns 1; missing required CLI output-directory information returns 2. A GUI-oriented executable built with `--windowed` does not provide a normal console for help and progress text. Use the Python command or the console build described below when command-line output is needed.

10. Windows installation and virtual environment

10.1 Runtime and build dependencies

Pillow, imported in Python as `PIL`, is required for visualization. `pip` is the package installer. The preview uses the `ezdxf` drawing frontend with a custom Tk Canvas backend. That frontend imports Pillow even when raster images are represented by outlines. Installing only `ezdxf` and `pyproj` therefore does not establish that the preview can start. [Pillow, Installation](#); [ezdxf, Setup](#)

Table 12. Complete component inventory for this application.

Component / package	Python import	Function / scope
Python 3.12, 64-bit, including Tcl/Tk	<code>sys</code> , <code>tkinter</code> , other standard-library modules	Source interpreter and GUI runtime; install on the build computer
<code>pip</code>	Run as <code>python -m pip</code>	Installs and checks packages; does not render drawings
<code>setuptools</code> , <code>wheel</code>	Build support tools	Maintain the environment's packaging support; not drawing renderers
<code>ezdxf==1.4.4</code>	<code>ezdxf</code>	Reads/writes DXF, represents CAD entities and supplies the drawing frontend
<code>pyproj==3.7.2</code>	<code>pyproj</code>	Supplies the PROJ interface, coordinate operations and packaged PROJ resources
<code>Pillow>=12.0,<13</code>	<code>PIL</code>	Image utilities imported by the drawing frontend; explicitly install for this preview
<code>numpy>=1.26,<3</code>	<code>numpy</code>	Numerical arrays used by <code>ezdxf</code> ; normally installed as its dependency
<code>fonttools>=4.61,<5</code>	<code>fontTools</code>	Font processing for CAD text; normally installed as an <code>ezdxf</code> dependency
<code>pyparsing>=3.0,<4</code>	<code>pyparsing</code>	Parsing facilities required by <code>ezdxf</code>
<code>typing_extensions>=4.6,<5</code>	<code>typing_extensions</code>	Runtime typing compatibility utilities required by <code>ezdxf</code>
<code>certifi</code>	<code>certifi</code>	Certificate bundle declared as a <code>pyproj</code>

Component / package	Python import	Function / scope
PyInstaller>=6.15,<7	PyInstaller	dependency; does not supply transformation grids Produces the application executable; install on the build computer
pyinstaller-hooks-contrib	Hooks consumed by PyInstaller	Supplies maintained collection rules for third-party modules, data and binaries
Four correctly named NTV2 .gsb files	Data files, not modules	Historical mainland datum corrections; supplied beside the source and embedded by the build below
ODA File Converter	Separate native executable	Required for DWG input, output and preview; not installed by pip

Column key (left to right): Component or pip package; Python import name; Function and installation scope.

Pip resolves declared dependencies automatically; the installation command nevertheless lists every runtime package above explicitly. Core ezdxf does not declare Pillow as a mandatory base dependency. Installing `ezdxf[drawing]` would pull additional drawing backends, including libraries that this application's custom preview does not use. Matplotlib, PyQt, PySide and PyMuPDF are unnecessary for this implementation. [ezdxf](#), [Setup](#)

Tkinter, Tcl/Tk, `venv`, `ensurepip`, `argparse`, `json`, `pathlib`, `threading`, `ctypes` and the other standard-library components are provided by the Python installation. **Do not run `pip install tkinter`, `pip install PIL` or `pip install venv`.** Select Tcl/Tk and pip when installing Python; the image package's install name is Pillow. [Python Software Foundation](#), [venv](#); [Pillow](#), [Installation](#)

The core packages are pinned to the API baseline used for application verification. Auxiliary version bounds permit compatible upgrades; they do not claim that every future package combination has been tested. The build record must retain the versions actually resolved by pip.

10.2 Create and activate the virtual environment

Place `script.py`, `README.md` and the four chosen grid files in a working folder, for example `C:\DOWNLOADS\cad-epsg-conversion`. Open **Windows Command Prompt** (`cmd.exe`) there. The following are shell commands, not Python statements for the `>>>` prompt:

```
cd /d "C:\DOWNLOADS\cad-epsg-conversion"
py -3.12 -m venv .venv
call .venv\Scripts\activate.bat
python -c "import sys; assert sys.prefix != sys.base_prefix"
python -c "import sys; print(sys.executable)"
```

The printed path should end in the project's `.venv\Scripts\python.exe`. Python creates the virtual environment; its activation script selects that environment in the current command shell. Activation usually adds `(.venv)` to the prompt. Every subsequent `python -m pip` command then targets this environment rather than an unrelated global installation. The `call` form also works when these commands are saved in a Windows batch file. [Python Software Foundation, venv](#)

If `py -3.12` cannot select an interpreter, install 64-bit Python 3.12 with its launcher, `pip` and `Tcl/Tk` components, or use the full path to that interpreter for the environment-creation command. An existing environment can be activated without creating it again.

10.3 Bootstrap pip, upgrade installation tools and install every runtime package

Run the following in the **activated environment**:

```
python -m ensurepip --upgrade
python -m pip install --upgrade pip setuptools wheel
python -m pip install --upgrade ^
    "ezdxf==1.4.4" "pyproj==3.7.2" "Pillow>=12.0,<13" ^
    "numpy>=1.26,<3" "fonttools>=4.61,<5" "pyparsing>=3.0,<4" ^
    "typing_extensions>=4.6,<5" certifi
python -m pip check
```

`ensurepip` bootstraps `pip` from Python's bundled resources; it does not obtain the newest `pip` release from the internet. The following `pip install --upgrade pip setuptools wheel` command upgrades the installation tools from the configured package index. Normal dependency installation requires access to that index or an equivalent local wheel repository. [Python Software Foundation, ensurepip; pip, Installation](#)

`--upgrade` requests the newest available version allowed by each requirement. The exact `ezdxf` and `pyproj` pins retain their specified versions; the bounded supporting packages can advance within their ranges. Quotes are necessary around requirements containing `<` or `>` because Command Prompt otherwise interprets those characters as redirection. Each caret `^` must be the last character on its line, with **no trailing spaces**.

The selected Python version and architecture allow the usual Windows binary wheels for these packages. If `pip` unexpectedly tries to compile a native package and fails, first confirm that the active interpreter is 64-bit Python 3.12 and that `pip` was upgraded. A compiler should not be assumed to repair a mismatched environment.

10.4 Check the visualization stack and start the program

`pip check` validates declared dependency relationships. It cannot detect an undeclared optional dependency such as Pillow in the base ezdxf installation. Import the **drawing frontend**, in addition to the top-level packages. Each import command must finish without an error; the following commands then print the installed versions:

```
python -c "import tkinter, ezdxf, pyproj"
python -c "import PIL, numpy, fontTools"
python -c "from ezdxf.addons.drawing import Frontend"
python -c "from ezdxf.addons.drawing import RenderContext"
python -c "import ezdxf; print('ezdxf', ezdxf.__version__)"
python -c "import pyproj; print('pyproj', pyproj.__version__)"
python -c "import pyproj; print('PROJ', pyproj.proj_version_str)"
python -c "import PIL; print('Pillow', PIL.__version__)"
python -m tkinter
python script.py
```

Close the small Tkinter demonstration window before proceeding to `python script.py`. In the application, open a representative DXF containing lines, arcs, text and hatches. Confirm that model space appears in the black preview, then test pan, zoom and Fit. A successful import alone does not verify that actual drawing content renders correctly.

For the coordinate runtime's diagnostic information:

```
python -c "import pyproj; pyproj.show_versions()"
```

Check the selected transformation through **Validate transformation**, then convert a drawing with known survey control. Python package checks do not establish that a source CRS is correct, that the proper grid was selected, or that ODA is installed.

10.5 Reactivation, shell syntax and leaving the environment

Each new Command Prompt needs activation before using the abbreviated commands in this document:

```
cd /d "C:\DOWNLOADS\cad-epsg-conversion"
call .venv\Scripts\activate.bat
python script.py
deactivate
```

Run `deactivate` after the application or build finishes. Closing the shell also ends its activation. An environment is a local development installation, not a portable distribution; recreate it rather than copying it to another machine. [Python Software Foundation](#), [venv](#)

Shell	Activation command	Continuation
Windows Command Prompt, used throughout this document	<code>call .venv\Scripts\activate.bat</code>	Caret ^ at the end of each continued line
PowerShell	<code>.\.venv\Scripts\Activate.ps1</code>	PowerShell backtick, or place the

Shell	Activation command	Continuation
		command on one line

Column key (left to right): Shell; Environment activation from the project folder; Continuation syntax.

The multi-line commands below are written for **Command Prompt**. Do not paste them unchanged into PowerShell. If PowerShell policy prevents activation, use Command Prompt with the documented `.bat` command. Calling an activation script from inside a Python subprocess cannot change the parent shell's environment; activation belongs in the shell before `python script.py`.

10.6 DWG support and ODA File Converter

Install a Windows-compatible ODA File Converter from the Open Design Alliance when DWG is required. It is a native format-conversion program with GUI and command-line interfaces. Neither `pip install ezdxf` nor installing a similarly named Python wrapper supplies its executable. [Open Design Alliance, ODA File Converter](#)

The script searches the explicit GUI/CLI setting, `ODA_FILE_CONVERTER`, `ODA_CONVERTER`, executable names on `PATH`, and supported Windows Program Files installation patterns. If detection fails, browse to the executable in Advanced settings or supply `--oda`.

Each ODA call receives an isolated copied input, requests `ACAD2018` output, disables recursion and enables auditing. There is no CAD-release selector in the application. ODA supplies the format bridge; PROJ performs the geodetic work. Native DXF input is handled through its document version where the resulting entities are supported. [ezdxf, ODA support](#)

11. Syntax compilation and standalone executable

11.1 Meaning and scope of a standalone build

The build below is intended to produce a **single Windows executable with all Python dependencies, the GUI runtime, PROJ resources and all four transformation grids embedded**. After validation, DXF opening, preview, coordinate conversion and DXF output require no separate Python, pip, Pillow or grid-file installation on the destination computer. Windows, normal system facilities and access to the user's drawing files are still required.

DWG input, output and preview require ODA File Converter separately. The current program therefore provides self-contained DXF operation, with an external dependency for DWG. A fully self-contained executable covering both DWG and DXF requires a compatible redistributable DWG runtime, its integration and clean-machine testing. No pip command or PyInstaller flag adds that engine to this application. [Open Design Alliance, ODA File Converter; PyInstaller, Operating mode](#)

Resource	Bundled content	Host requirement
Python interpreter and imported	Yes	No separate Python installation

Resource	Bundled content	Host requirement
standard-library modules		
Tcl/Tk, Tkinter and application GUI	Collected by PyInstaller's Tk hooks	Compatible Windows desktop environment
ezdxf, Pillow/PIL, NumPy, FontTools and supporting packages	Collected with their modules and required binaries	No pip installation
pyproj, PROJ binaries and coordinate-system database	Collected with pyproj resources and hooks	No separate PROJ installation
Four <code>.gsb</code> grids supplied at build time	Explicitly embedded by <code>--add-data</code>	No external grids for the corresponding operations
DWG decoding/encoding	No	ODA File Converter, detected or selected explicitly
Custom fonts, Xrefs, underlays and other drawing-specific assets	Not automatically embedded	Provide the assets needed by the drawing; font substitution can affect preview appearance
Output CAD files and JSON reports	Created at runtime	A writable output folder
Temporary runtime and conversion files	Created as required	Available temporary disk space and permissions

Column key (left to right): Capability or resource; Included by the documented one-file build; Destination-computer requirement.

One-file packaging describes delivery, not execution entirely inside the executable. PyInstaller extracts bundled resources into a temporary runtime directory and normally removes them when the application exits. This behaviour can affect startup time and the grid paths recorded in a report. [PyInstaller](#), [Runtime information](#)

11.2 Syntax verification and build-tool installation

Build a Windows executable **on Windows**, using the intended 64-bit Python 3.12 environment. A Linux build produces a Linux executable; PyInstaller is not a Windows cross-compiler. Start from a clean project environment containing the required packages from Section 10, then keep it activated:

```
python -m py_compile script.py
python -m pip install --upgrade ^
    "PyInstaller>=6.15,<7" pyinstaller-hooks-contrib
python -m pip check
python -m PyInstaller --version
```

`py_compile` checks source syntax and writes bytecode in `__pycache__`. It does **not** produce

a standalone executable, collect dependencies or exercise the GUI. PyInstaller supplies the packaging step. The hooks package provides additional third-party collection rules and should be installed together with the build tool. [Python Software Foundation, py_compile](#); [PyInstaller, Hooks](#)

Keep this environment limited to the application requirements. Broad collection of ezdxf modules can otherwise pull in optional backends that happen to be installed. A build log can mention unavailable optional drawing backends; investigate warnings affecting this program's Tk/Pillow/font/PROJ path rather than installing unrelated GUI frameworks merely to silence every optional-import warning.

11.3 One-file GUI build with embedded grids

Before building, the current project folder must contain these files:

File	Build role
<code>script.py</code>	Main application, version 1.0
<code>pt73_e89.gsb</code>	Selected Datum 73 to ETRS89 grid
<code>ptLX_e89.gsb</code>	Selected Lisbon datum to ETRS89 grid
<code>ptED_e89.gsb</code>	Selected ED50 to ETRS89 grid
<code>ptLB_e89.gsb</code>	Selected Lisbon 1890 to ETRS89 grid

Where the DGT files `D73_ETRS89_geo.gsb` and `DLX_ETRS89_geo.gsb` are used, apply the datum-specific local filename aliases explained in Section 5 before packaging. A filename does not change a grid's source datum, transformation direction or precision. Preserve the chosen files' provenance and checksums.

Run in the activated environment, from that folder:

```
python -m PyInstaller --noconfirm --clean --onefile ^
--windowed --noupv ^
--name script --collect-all ezdxf --collect-all pyproj ^
--collect-all PIL --collect-all fontTools ^
--add-data "pt73_e89.gsb:." --add-data "ptLX_e89.gsb:." ^
--add-data "ptED_e89.gsb:." --add-data "ptLB_e89.gsb:." ^
script.py
```

The output is `dist\script.exe`. Each quoted `--add-data` value uses `SOURCE:DESTINATION`; `.` places the grid at the bundled resource root. All four files must exist when building. Keep every caret at the end of its line without trailing spaces. [PyInstaller, Usage](#)

Table 13. Build options and resource collection.

Option / mechanism	Purpose
<code>--onefile</code>	Deliver one application executable containing the selected resources
<code>--windowed</code>	Use a GUI executable without a normal console window
<code>--name script</code>	Produce <code>script.exe</code>
<code>--collect-all ezdxf</code>	Collect CAD modules, resources and package

Option / mechanism	Purpose
	binaries
<code>--collect-all pyproj</code>	Collect pyproj modules, resources and binaries, with its dependency hooks
<code>--collect-all PIL</code>	Collect Pillow using its import name , including image plugins and package binaries
<code>--collect-all fontTools</code>	Collect FontTools using its case-sensitive import name, including dynamically accessed modules
Standard/contributed hooks	Handle additional dependency resources, NumPy native libraries and Tcl/Tk collection
Four <code>--add-data</code> arguments	Embed the actual NTV2 files used by the application
<code>--noupX</code>	Do not use UPX executable compression
<code>--clean</code>	Clear the build cache before packaging
<code>--noconfirm</code>	Permit replacement of the build output without an interactive confirmation

Column key (left to right): Option or mechanism; Purpose.

Pillow’s pip name is `Pillow`; its import and PyInstaller collection name is `PIL`. Similarly, pip installs `fonttools`, while collection uses `fontTools`. These names are intentional. Collection flags request the resources; the acceptance checks below establish whether a particular produced executable contains everything required on the target machine. [PyInstaller](#), [Hooks](#)

11.4 Embedded grids, overrides and deployment contents

For the embedded-grid deployment, distribute `dist\script.exe` with the English documentation and applicable grid attribution. The README supplies operating instructions and provenance; it is not loaded as a runtime dependency. External `.gsb` files and the `.venv` folder are **not required** for the embedded operations.

Leave **Advanced > NTV2 grid folder** blank to use the default search. In a frozen application, matching external grids beside `script.exe` have priority over embedded resources. An explicitly selected grid folder is exclusive; a stale folder selection can therefore prevent use of otherwise valid embedded grids. Do not set a development-machine grid path on the destination computer.

The embedded files are fixed at build time. To deploy different embedded grids, rebuild with the chosen files. For an intentional external override, place the correctly named grids beside the executable or select a complete grid folder, and check the report’s resolved paths and hashes. In either case, preserve DGT attribution and the grid provenance described in this paper.

Embedding the grid data has no effect on ODA discovery. Adding an arbitrary ODA executable to a build’s data files is not a verified portable DWG deployment; its runtime dependencies and the application’s discovery behaviour must also be addressed. The documented build uses an independently installed ODA converter for every DWG path.

11.5 Console executable for command-line operation and diagnostics

For an executable that displays command-line help, errors and progress, omit `--windowed` while retaining **all package collection and embedded-grid options**:

```
python -m PyInstaller --noconfirm --clean --onefile --noupx ^
  --name script --collect-all ezdxf --collect-all pyproj ^
  --collect-all PIL --collect-all fontTools ^
  --add-data "pt73_e89.gsb:." --add-data "ptLX_e89.gsb:." ^
  --add-data "ptED_e89.gsb:." --add-data "ptLB_e89.gsb:." ^
  script.py
dist\script.exe --help
dist\script.exe "survey.dxf" --output-dir "output" ^
  --source-epsg 27493 --target-epsg 3763 --format dxf
```

Both build commands produce `dist\script.exe`; choose the intended variant before distribution because the second build replaces the first. The console variant can also launch the GUI when run without input drawings. If a windowed build fails before displaying its GUI, a console build from the same environment helps expose startup exceptions.

11.6 Reproducibility and clean-machine acceptance

Capture the resolved environment after installing both runtime and build dependencies:

```
python -m pip freeze --all > build-requirements.txt
python -c "import pyproj; pyproj.show_versions()" ^
  > build-environment.txt
python -c "import sys; print(sys.version)" ^
  >> build-environment.txt
python -c "import platform; print(platform.platform())" ^
  >> build-environment.txt
python -c "import platform; print(platform.machine())" ^
  >> build-environment.txt
python -m PyInstaller --version >> build-environment.txt
certutil -hashfile dist\script.exe SHA256
```

Retain the build files, Windows/Python architecture, package versions, executable hash and grid hashes with the release record. `pip freeze --all` records installed versions, including pip; it is an environment snapshot rather than a dependency solver's lockfile. It does not capture the interpreter installer, OS, external ODA installation, custom fonts or grid data. On a separate build machine, recreate and activate the environment, bootstrap pip, then run `python -m pip install -r build-requirements.txt` and supply the recorded grids before repeating the build. [pip](#), [pip freeze](#)

The script isolates the external converter's library-search environment when launched from a frozen application, including restoration of the Windows DLL search directory. This is necessary because the bundled Python runtime and the external ODA process can require different libraries. [PyInstaller](#), [Common issues](#)

Acceptance check	Expected evidence
Run only the executable, without Python, a virtual environment or external <code>.gsb</code> files	GUI opens with the input controls visible and the preview available

Acceptance check	Expected evidence
Open a DXF containing lines, curves, text and hatches Convert known Datum 73 control to EPSG:3763 with the grid folder blank Check reverse and other required datum paths	The black preview renders representative content; Fit, pan and zoom work Output and report_3763.json are produced; report identifies the embedded grid and its expected hash Selected operations resolve the intended embedded grid files and agree with control within the project's acceptance criteria
Test the local-grid DXF workflow without an internet connection	No package or grid download is needed for the supplied operations
Use paths containing spaces and a writable output folder Review the output in the intended CAD application	CAD output and report creation succeed Relevant geometry, layers, text and complex entities meet the documented scope and tolerances
Install/select ODA and test DWG input and output separately	ODA runs successfully and the resulting files reopen in the intended CAD software

Column key (left to right): Acceptance check on a clean Windows machine or VM; Expected evidence.

These are deployment acceptance steps, not a claim that a particular Windows executable has already passed them. Source syntax, package imports and coordinate tests cannot replace execution of the frozen program on the target platform. A successful native build plus these checks is the evidence for a working standalone DXF distribution.

12. Reports, reproducibility and troubleshooting

12.1 Output names and report lifecycle

A default input **survey.dxf** produces the following file, according to the selected output format:

- DXF output: **survey_EPSG3763.dxf**.
- DWG output: **survey_EPSG3763.dwg**.

The report is named **report_3763.json**. Selecting another target changes the report name to **report_<target EPSG>.json**.

Each run replaces the report for that target in the selected output folder, independently of the CAD overwrite setting. Archive the report elsewhere if separate run records are required. Existing CAD outputs are replaced only when overwrite is enabled. Output/input collisions and duplicate planned output names are rejected before processing.

If a batch stops on an error or cancellation, completed drawings remain available. The report records the batch state and completed files when processing has started. An initial validation failure can occur before report creation. Detailed per-file entity counters describe completed

drawings; the failed current drawing is identified through the error entry rather than a complete partially accumulated entity inventory.

12.2 Report contents

Table 14. Main report information.

Report area	Information
Application/job	Name, version, selected settings and input/output paths
Execution	Start/finish UTC timestamps, completed count and batch status
Coordinate operation	Source/target identifiers, names, operation description and pipeline
Grid provenance	Actual filename, resolved path, SHA-256, byte size, internal version, datum identifiers, direction and bounds
CAD processing	Transformed, approximated, local-affine, exploded and unresolved counters
Entity issues	Severity, issue code, message, layout, entity type and handle
Read/write assurance	Input audit/recovery and output reopening/audit/count checks
Failure/cancellation	Error type, message and current input path where available

For a local-grid chain, `accuracy_metres` is deliberately `null`. The script does not replace it with a guaranteed centimetre value or automatically insert DGT's validation RMSE. For generic PROJ chains, reported stage accuracy information is descriptive metadata; it is not the covariance propagation described in Section 7.1.

After using the DGT files under the application aliases, look for the actual internal version IGP2011 and the intended file hash. The name `pt73_e89.gsb` alone does not identify which set of binary corrections was used.

12.3 Temporary files and cancellation

DWG exchange and output verification use isolated temporary directories. Completed CAD output is staged in the destination filesystem before publication. Normal completion, handled errors and cooperative cancellation clean the temporary working data.

Cancellation is checked between processing steps and entities. Some expensive operations must return before the next check; cancellation need not be instantaneous. When ODA is active, the application attempts to stop its process tree and clean the associated workspace. Abrupt operating-system termination is outside the normal cleanup sequence.

12.4 Diagnostic guide

Table 15. Common symptoms and checks.

Symptom	Issue / next check
No module named PIL, or preview dependency error from source	Activate the project environment, install Pillow and the complete Section 10 runtime requirements, then repeat the drawing-frontend import check
Preview dependency error from <code>script.exe</code>	Rebuild with Pillow installed and <code>--collect-all PIL</code> ; installing pip packages beside a frozen executable does not repair its bundle
Tkinter demonstration fails before the program starts	Check the Python installation's Tcl/Tk components; Tkinter is not a pip package
A Command Prompt command reports redirection or caret errors	Quote version constraints and remove trailing spaces after carets; use the specified shell
Executable cannot use its embedded grid	Clear the explicit grid-folder setting, check for unintended external overrides and inspect the recorded grid path
Required NTV2 grid is missing	Check the expected alias filename and the exclusive/default search rules
Original DGT downloads are present but not found	Supply compatible copies as <code>pt73_e89.gsb</code> and <code>ptLX_e89.gsb</code>
Unexpected datum or units in grid header	A file was assigned to the wrong datum family; check binary provenance
Grid coverage error	Check the declared source CRS, metre/degree interpretation and actual geographic extent
Very large or implausible displacement	Check source datum, false origin, axis order and source drawing units
EPSG:2963 result is inconsistent	Check X=southing/Y=westing and the historical Bessel–Bonne realisation
DWG conversion cannot start	Check ODA installation, executable path and operating-system compatibility
Output exists	Select another folder/suffix or deliberately enable overwrite
Two drawings create one output name	Rename one input or process them separately
Strict mode stops a block/dimension	Inspect the reported composite condition; unsupported content cannot be assumed converted

Symptom	Issue / next check
Dimension text differs from a measured target length	Rendered text was transformed, not recalculated as a target-system measurement
Preview looks incomplete	Read preview notices; display limits do not define conversion limits
Preview still shows source coordinates after conversion	The preview is the selected original input; open the output separately
No CLI text from <code>script.exe</code>	Use the console build or run the Python source
Elevations appear unchanged	Expected for horizontal conversion with Keep Z enabled; no geoid correction is applied

Column key (left to right): Symptom; Likely issue or next check.

13. Discussion and conclusions

The principal geodetic decision is the correct identification of the source system. A precise numerical engine applied to the wrong datum, projection, units or axes can produce a consistent but incorrect drawing. For mainland work requiring the national target system, the documented workflow centres on EPSG:3763 and uses the corresponding historical-datum grid when necessary.

Grid transformations represent spatially varying datum differences that a single national translation or similarity transformation cannot fully describe. Their usefulness depends on the quality and coverage of the underlying observations, not simply on file size. Recording the grid's version and hash makes a transformation reproducible and permits later comparison with independent control.

CAD introduces a second problem beyond geodesy: the target geometry may not belong to the same analytical or parametric object class as the source. The application addresses this through explicit geometric strategies and records approximations and unresolved conditions. It does not claim universal exact preservation of every proprietary object, embedded coordinate payload or associative editing relationship.

A defensible project deliverable combines three forms of evidence: a documented coordinate-operation chain, independent agreement with suitable control points, and inspection of the CAD objects that carry engineering meaning. The JSON report supports the first and records structural processing information; the remaining evidence comes from the project's survey and CAD review.

14. Bibliography

Titles of Portuguese-language resources are translated into English below. Institutional web pages without a stated publication date are identified as *n.d.* Online documentation was consulted on 16 September 2026. The equations are explanatory formulations in this document's notation; citations distinguish institutional specifications, methodological literature and software documentation.

14.1 Portuguese reference systems and DGT specifications

1. **Direção-Geral do Território (DGT).** n.d. [Reference systems](#). Geodesy documentation, Portugal. Primary overview of mainland and island reference systems and national/island vertical datums. Portuguese-language institutional resource.
2. **Direção-Geral do Território (DGT).** n.d. [PT-TM06/ETRS89](#). Definition, establishment and projection parameters of the mainland national system. Principal institutional reference for the EPSG:3763 workflow. Portuguese-language institutional resource.
3. **Direção-Geral do Território (DGT).** n.d. [Coordinate transformations: mainland Portugal](#). NTv2 grid construction and validation, downloadable Datum 73/Datum Lisboa grids, and parameter-transformation alternatives. DGT states a Creative Commons Attribution 4.0 International licence for the grid work. Portuguese-language institutional resource.
4. **Direção-Geral do Território (DGT).** n.d. [PTRA08-UTM/ITRF93](#). Reference-system establishment and UTM parameters for the autonomous regions. Portuguese-language institutional resource.
5. **Direção-Geral do Território (DGT).** n.d. [Datum 73](#). Historical mainland network, origin and Hayford/Gauss–Krüger parameters. Portuguese-language institutional resource.
6. **Direção-Geral do Território (DGT).** n.d. [Datum Lisboa](#). Historical mainland system and Local Lisbon Triangulation context. Portuguese-language institutional resource.
7. **Direção-Geral do Território (DGT).** n.d. [Bessel Datum Lisboa](#). Bessel–Bonne parameters, west/south coordinate directions and the qualification concerning historical polynomial derivation. Portuguese-language institutional resource.
8. **Direção-Geral do Território (DGT).** n.d. [ED50: European Datum 1950](#). Datum/ellipsoid context and UTM representations relevant to Portuguese data. Portuguese-language institutional resource.
9. **Direção-Geral do Território (DGT).** n.d. [Geoid model](#). GeodPT08, the relationship between ellipsoidal and orthometric heights, and available geoid datasets. Portuguese-language institutional resource.
10. **Instituto Geográfico Português (IGP), hosted by DGT.** 2009. [Transformation parameters for mainland Portugal](#). Parameter sheet dated July 2009; degree-2 polynomial, Bursa–Wolf and Molodensky parameters and residual statistics. Portuguese-language PDF.
11. **Direção-Geral do Território (DGT).** n.d. [Formula sheet: second-degree polynomial transformation](#). Normalized-coordinate equations and coefficient definitions. Portuguese-language PDF.
12. **Direção-Geral do Território (DGT).** n.d. [Formula sheet: Molodensky transformation](#). Geographic-coordinate and ellipsoidal-height equations, ellipsoid differences and curvature radii. Portuguese-language PDF.
13. **Direção-Geral do Território (DGT).** n.d. [Formula sheet: Bursa–Wolf transformation](#). Small-angle transformation matrix, parameter definitions and rotation-convention

qualification. Portuguese-language PDF.

14.2 Scientific and geodetic methodology

14. **Gonçalves, J. A.** 2009. [Conversions of national coordinate systems to ETRS89 using grids](#). Paper presented at the VI National Conference on Cartography and Geodesy, CNCG 2009. Faculty of Sciences, University of Porto. Portuguese-language conference paper; the project-supplied PDF was examined, including its independent validation results.
15. **Gonçalves, J. A.** n.d. [Coordinate transformations in Portugal](#). Faculty of Sciences, University of Porto. Portuguese-language technical resource describing four historical-datum NTv2 grids and their use with PROJ; consulted through the project-supplied PDF copy.
16. **International Association of Oil & Gas Producers (IOGP).** 2019. [Geomatics Guidance Note 7, Part 2: Coordinate Conversions and Transformations including Formulas](#). Report 373-7-2, September 2019 edition. Reference for operation terminology and mathematical methods, including grid interpolation and inverse transformations. This citation identifies the particular edition consulted.
17. **Snyder, J. P.** 1987. [Map Projections—A Working Manual](#). United States Geological Survey Professional Paper 1395, ix + 385 pp. Washington, DC: United States Government Printing Office. DOI: 10.3133/pp1395. Classical further reading on ellipsoidal map projections, their parameters and distortion.
18. **Joint Committee for Guides in Metrology (JCGM).** 2008, corrected 2010. [Evaluation of Measurement Data—Guide to the Expression of Uncertainty in Measurement](#). JCGM 100:2008. Sections 5.1–5.2 provide the general first-order framework for uncertainty propagation, including correlated inputs.

14.3 Coordinate and CAD software

19. **PROJ contributors.** n.d. [Horizontal grid shift](#). PROJ operation documentation. Grid-based horizontal corrections, supported grid formats and optional-grid behaviour.
20. **PROJ contributors.** n.d. [Geodetic TIFF grids](#). PROJ grid-format specification, especially longitude-offset sign conventions and their relationship to NTv2.
21. **PROJ contributors.** n.d. [Geodetic to Cartesian conversion](#). PROJ operation documentation. Geographic/geocentric coordinate representations and Cartesian axes.
22. **PROJ contributors.** n.d. [Transverse Mercator](#). PROJ projection documentation. Mathematical formulation, parameters and precise numerical implementation.
23. **PROJ contributors.** n.d. [Helmert transform](#). PROJ operation documentation. Three-/seven-parameter and time-dependent forms, units and rotation conventions.
24. **pyproj contributors.** n.d. [Transformer API](#). pyproj documentation. Coordinate-operation construction, `always_xy`, operation availability and accuracy metadata.
25. **ezdxf contributors.** n.d. [ODA File Converter support](#). ezdxf 1.4.4 documentation. External converter integration and DWG/DXF exchange context. The application uses its own controlled subprocess workflow.

26. **Open Design Alliance.** n.d. [ODA File Converter](#). Product and download documentation, including supported command-line conversion inputs.

14.4 Python environment and executable packaging

27. **Python Software Foundation.** n.d. [venv—Creation of virtual environments](#). Python standard-library documentation. Environment isolation, direct interpreter invocation and platform-specific activation.
28. **Python Software Foundation.** n.d. [py_compile—Compile Python source files](#). Python standard-library documentation. Bytecode compilation and syntax checking.
29. **PyInstaller development team.** n.d. [Using PyInstaller](#). Build options, one-file/windowed execution and collection of package resources.
30. **PyInstaller development team.** n.d. [Runtime information](#). Resource locations, `sys.frozen`, `sys._MEIPASS` and bundled data-file handling.
31. **PyInstaller development team.** n.d. [Common issues and pitfalls](#). In particular, launching external programs from frozen applications and library-search-path isolation.
32. **Pillow contributors.** n.d. [Basic installation](#). Pillow installation, supported binary distributions and the distinction between the Pillow distribution and PIL import namespace.
33. **ezdxf contributors.** n.d. [Setup and dependencies](#). Core installation requirements and optional extras for drawing backends. Application-specific preview requirements were also checked against the installed ezdxf 1.4.4 frontend.
34. **Python Software Foundation.** n.d. [ensurepip—Bootstrapping the pip installer](#). Bootstrapping pip from bundled components and the scope of its upgrade option.
35. **pip contributors.** n.d. [Installation](#). Supported pip installation and upgrade procedures.
36. **PyInstaller development team.** n.d. [How the One-File Program Works](#). Runtime extraction and the relationship between packaged application resources and the host operating system.
37. **PyInstaller development team.** n.d. [Understanding PyInstaller Hooks](#). Analysis hooks, package data, native libraries and the role of `pyinstaller-hooks-contrib`.
38. **pip contributors.** n.d. [pip freeze](#). Recording installed package versions in requirements format and the distinction between an environment snapshot and a solver lockfile.